

Web Development Technologies

COSC2276 / COSC3134 Semester 2 2026

01/ Tuesday 21.07.2026

Course Overview & C# - Part 1

SEGMENT 1

COURSE OVERVIEW & HOUSEKEEPING

Topics

- Course Overview & Housekeeping
- .NET v. 10.0
- C# v. 14.0 (Part I)
- JSON processing in C#

Staff

- Course Coordinator: Shekhar Kalra
shekhar.kalra@rmit.edu.au
- Lecturer & Tutor: Matthew Bolger
matthew.bolger2@rmit.edu.au
- Tutor: Liam Pietralla
liam.pietralla@rmit.edu.au
- Tutor: Jeffry Susanto
jeffry.susanto@rmit.edu.au

Consultation Protocols

- Consulting Lecturer: Matthew Bolger
 - General and subject related material
 - Assignment related queries
 - Marks, Extension and other enquiries
 - Via Email and Microsoft Teams to matthew.bolger2@rmit.edu.au
- Communication
 - Canvas Discussion Forum
 - Email and Microsoft Teams

Getting the best out of this course

- Recipe for Success

- Practise
- Practise
- Practise



- Programming skills come with practice not just thinking that you understand the concepts.
- It is RMIT policy that you are responsible for and regularly back up your work.

Software for this course

- Not available in labs
- You must have a laptop
- You must download and install your own software, as mentioned on the course guide for this subject.
- Details next slide→

Software for this course

1. IDE / Text Editor

- Windows users:
Visual Studio 2026 Community Edition +
ASP.NET and web development Workload
OR
.NET 10.0 SDK + Visual Studio Code + C# Dev Kit Extension
- macOS and Linux users:
.NET 10.0 SDK + Visual Studio Code + C# Dev Kit Extension

2. SQL Server Client

- Windows, macOS and Linux users:
Visual Studio Code + SQL Server (mssql) Extension

*NOTE: Check install details on
Canvas → Modules → Pre-reading →
Software Installation page*

ATTEND LECTORIAL & LAB(S)

- Lecture slides will only point to the theory/concepts and references for the daily material
- Just reading lecture notes and not attending lectorial/lab will lead to poor outcome(s)
- You must attend lectorial and labs for completing your learning process.
- Each lectorial and lab will cover code examples to emphasise the concept covered in the lecture slides

.NET 10.0 Framework

.NET

- What is .NET?
- What is it used for?
- What is the target market?
- Enterprise development

“An execution and development platform for building apps for Windows, Windows Phone, Windows Server, Windows Azure.”

“.NET is a platform for software development that can be used to create: web applications, web services, console applications, windows desktop applications, mobile applications.”

“Mature enterprise level development . Used by the Australian Government and most of our large banking corporations. Its market share is constantly increasing offering great employment opportunities.”

“The most chosen frameworks for enterprise level development are .NET & JEE”

.NET

“.NET can help you build you multi-lingual applications for multiple devices quickly and easily.”

- Global reach
- Interoperability
- Secure

“Through the use of an intermediate language, and its ability to easily integrate with web services, applications are decoupling allowing complex multi-platform applications to act like a single application.”

- Rapid best practice development

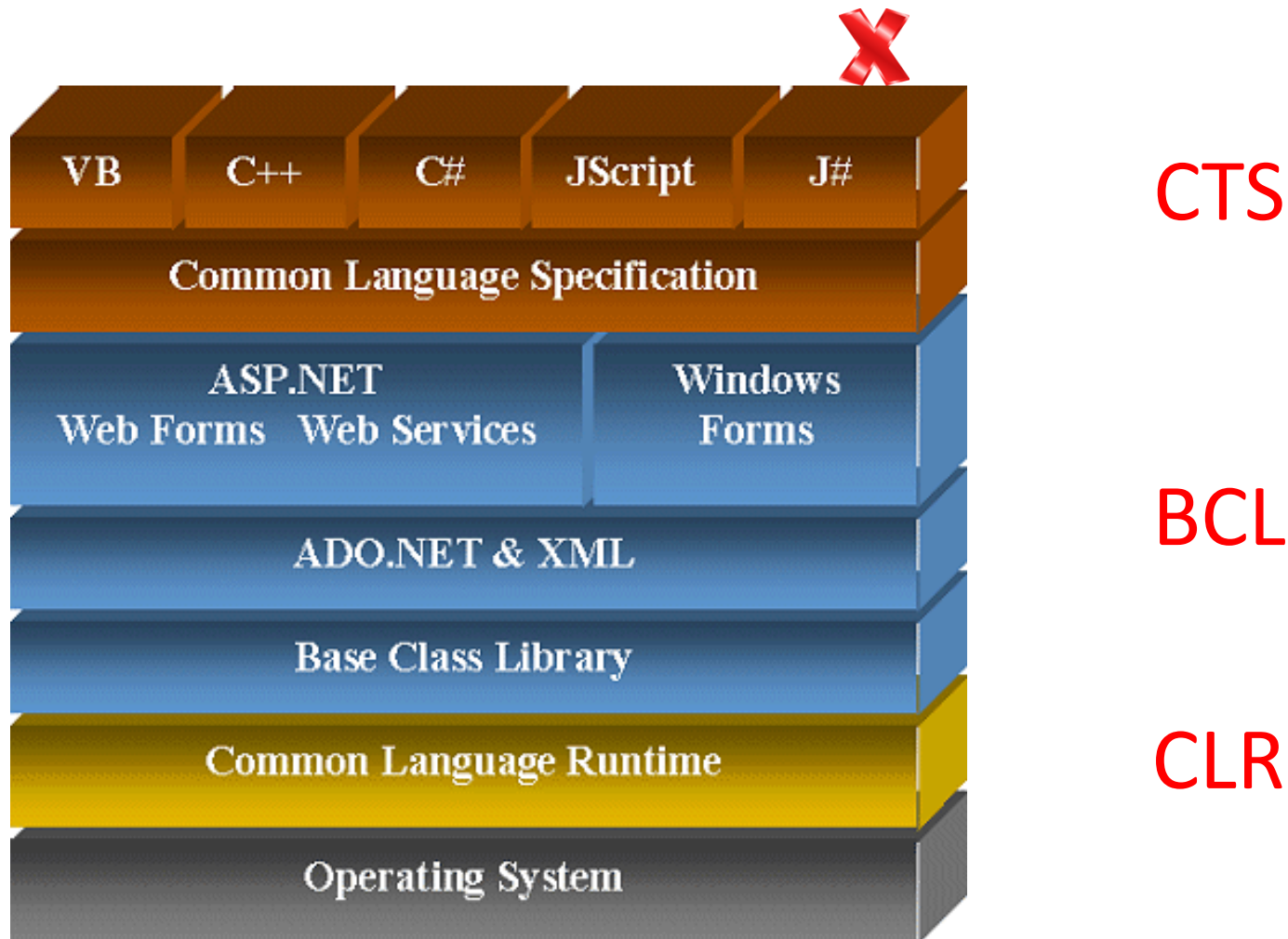
“.NET is built around a robust security model.”

“Like all frameworks many complex tasks are simplified and many best practices have been implemented on your behalf.”

.NET and .NET Core

- **.NET Framework 4.8:** a full framework for everything related to .NET, but lacks libraries & runtime optimized for multi-platform (Windows, Linux and Mac) and cloud deployments
- **.NET 10.0** on the other hand is a modern implementation of .NET Framework which is optimised for multi-platform and cloud deployments
 - <https://dotnet.microsoft.com/download>

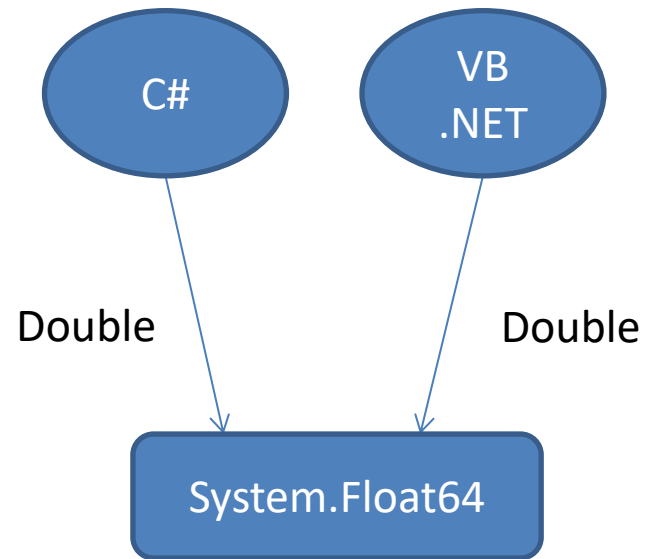
.NET (Components)



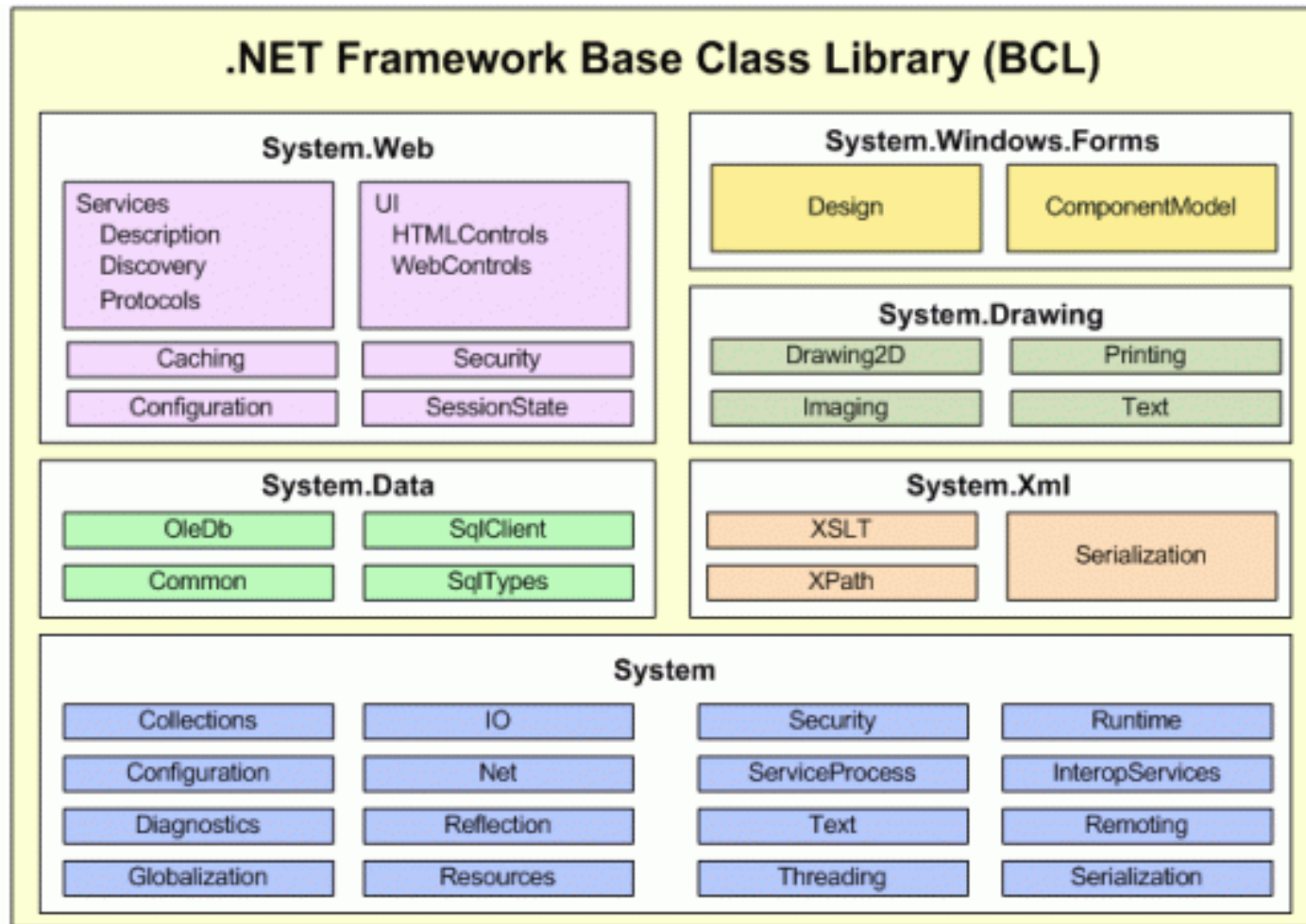
.NET – Common Type System

Some .NET built-in types:

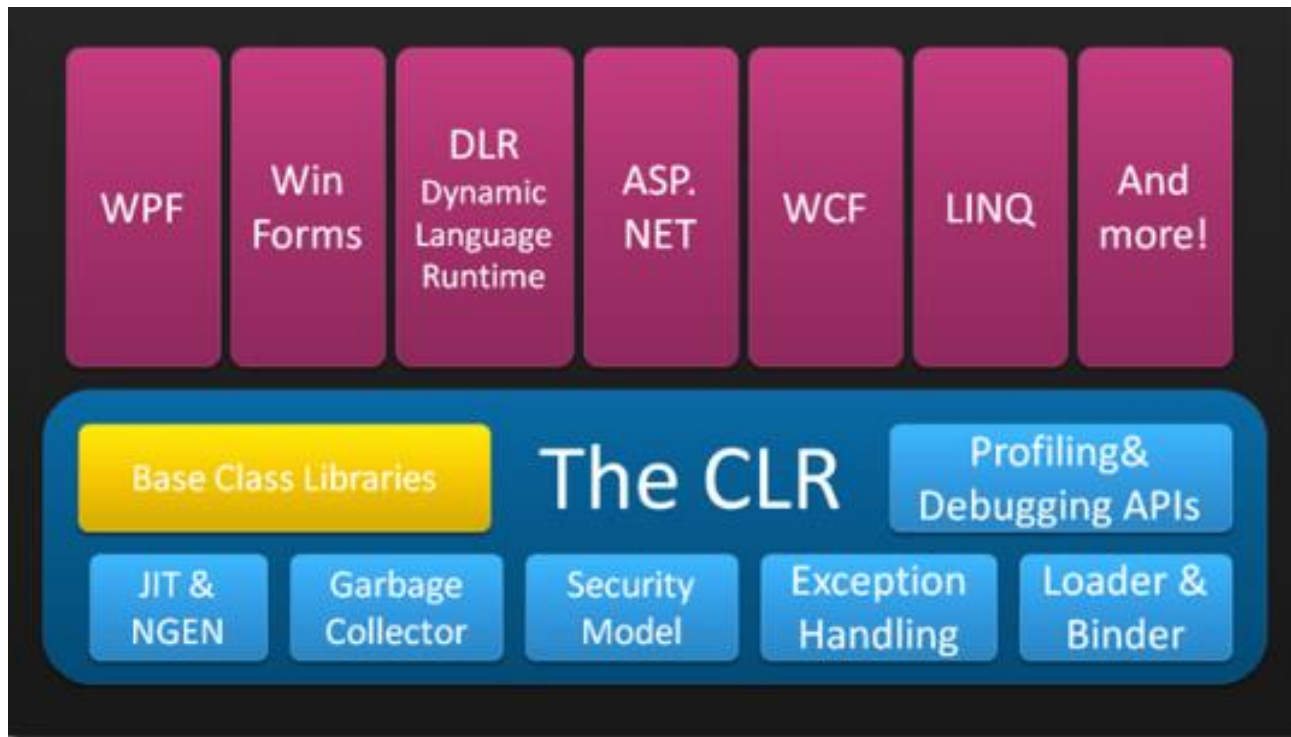
- **System.Object**
- System.Int16
- System.UInt16
- System.Int32
- System.UInt32
- System.Int64
- System.UInt64
- System.Char
- System.Single
- System.Double
- System.Boolean
- System.Decimal
- System.String



.NET (Base Class Libraries)



.NET (Common Language Runtime)



CTS

CLR

.NET (Compilation Process)

- Two stage compilation process
- Source code → Compiles → CIL (CTS)
- CLR (CIL) → JIT → Machine Code → executes

SEGMENT 2

BASICS OF C#

C# – Version History

- 1.0 - came with .NET framework 1.1
- 2.0 - came with .NET framework 2.0
- 3.0 - came with .NET framework 3.5
- 4.0 - came with .NET framework 4.0
- 5.0 - came with .NET framework 4.5 and 4.5.1
- 6.0 - came with .NET framework 4.6
- 7.0 - came with .NET framework 4.7
- 8.0 - came with .NET framework 4.8
- 9.0 - came with .NET 5.0
- 10.0 - came with .NET 6.0
- 11.0 - came with .NET 7.0
- 12.0 - came with .NET 8.0
- 13.0 - came with .NET 9.0
- 14.0 - came with .NET 10.0

C# – Introduction

- In this part of lecture one, we will:
 - Introduce C#
 - OO in C#
 - Strings
 - Error handling
 - Sealed class
 - Interface

C# – Introduction

- C# is an object oriented programming language that fully supports the functionality of the .NET framework. Other languages can also be used to program .NET. Visual Basic .NET is particularly popular
- C# can be used to develop windows applications, web applications, using ASP.NET (which we cover later), web services, and other technologies.
- C# is descended from the C and C++ programming languages.
- It was created by *Anders Hejlsberg*
- It is considered a sibling of Java - Java programmers will be immediately familiar with much of the C# language.

C# – Basics

- C# is very similar to many other programming languages, such as Java or C/C++. It has:
 - Primitive types: int, bool, char, float, double etc.
 - Constants
 - Declaration and assignment of variables
 - Expressions, operators, and operands: * + - % ++ etc.
 - Control structures: branching, looping
 - Comments

C# – Types (Value / Reference)

- In C# there are two types, value types and reference types. Value types are primitives, such as int and double. Reference types are the “classes” of other object oriented languages.
- While value types can be user defined (using structs this concept will be covered during tutorial session), this is not common.
- On the other hand, most C# programming is done by creating user defined reference objects (and using the many reference types available in the base framework).

C# – Object Type

- All objects in C# inherit from the System.Object type (even if it is not explicitly stated). Therefore all objects inherit the methods of System.Object:
 - Equals()
 - GetHashCode()
 - GetType()
 - ToString()
- Of course, these methods can be overridden in derived classes.

C# – Boxing & Unboxing

- The process of converting a primitive to an object is called {boxing}. The reverse process is called {unboxing}.
- Boxing usually occurs automatically. Unboxing must be called explicitly. Boxing copies the value of a primitive to an object that is created on the heap.
- You probably won't need to use boxing, but you should be aware that it occurs.

C# – Boxing & Unboxing

- For example, when you pass a primitive to the WriteLine method:

```
int num = 4;  
Console.WriteLine (num);
```

- The num primitive is boxed into an object. The ToString() method is then called, and its result passed to the WriteLine() method. This boxing occurs automatically.

C# – Classes & Objects

- A class represents an idea. An object, which is instantiated from a class, represents a concrete example of that idea.
- Classes have members: fields and methods. They also have constructors, properties, and events, all of which are considered special types of methods.
- Defining a class in C# is very similar to Java.

C# – A typical class

- Remember your java approach to class declaration
- C# is the same; the class variables hidden, and we create public methods to access them. This is a typical object oriented approach.
- Why?

C# – Changing Ref/Value

- A value type is passed to a method by value, that is, a copy is given to the method.
- Reference types, are passed by reference. If you want to pass a 'value type' i.e. primitive as a reference, use the keyword “ref” or “out” before both the declaration and call to the method.
- What advantage does this give you?

C# – Constructors

- The use of constructors is identical to that in Java.
- If no constructor is provided in a class, a default constructor (that takes no parameters) is created.
- In this example we also see the use of overloading. This means that multiple constructors can be written, each with a different signature.
- Any specified constructor can be used to instantiate an object.

C# – Properties

- Properties are a new approach to object oriented programming introduced by C#. Properties provide seemingly direct access to object variables.
- However it is more intuitive for a client to directly access a variable. Properties specify `get()` and `set()` methods but hide them from the client.
- The client accesses the property as if it were accessing a class variable directly.
- In the next lecture, we will learn new ways of writing these properties.

C# – Static

- In C# it is possible to have a
 - Static member, method, constructor and a class
- To do this the keyword “static” is used, such as on the Main() method -this method can be called without instantiating an object.
- Class methods are normally invoked by using the name of the class itself, then the name of the method.
- The best practice for static class variables is to set the visibility of the static variable to private, and use public static properties to access it.

C# – Static

- It is possible to have static classes in C# as well!
- Please note, it is poor programming practice to use static unless there is a good reason for doing so.
- The use of static is overused, particularly by poor programmers.
- Poor use of static breaks one of the major tenants of Object Oriented programming of ‘encapsulation’ and often leads to badly written code and lots of ‘bugs’.

C# – Visibility Modifiers

- C# uses the following levels of visibility:
 - public : available to any method
 - private: only available to methods of the owning class
 - protected: only available to methods of the owning class and any class that inherits from it
 - internal: only available to methods of classes in the assembly (covered *later in the course*)
 - protected internal: read as protected OR internal

C# - Strings

- The declaration of a string is straightforward using a string literal:
 - `string message = "error occurred";`
- Here the keyword `string` is actually an alias for the class `System.String`.
- There are many useful methods available to the `String` class, including `Compare()`, `Concat()`, `Equals()`, `Split()`, `Substring()`, `ToLower()`, `ToUpper()`, and `Trim()`.

C# - Strings

- The ToString() method (of the Object class) can be used to return a string representation of an object. This method is often overwritten.
- When non-strings values or objects are given to the Console.Write method, the string representation of these is printed by calling the ToString() method on them. (If you examine the documentation you will see the many overloaded versions of Console.Write.)

C# - Strings

- Other facts about C# strings:
- The String class in C# is sealed (SEE NEXT SLIDE) . This means that it cannot be derived from (obviously you can also make any custom class sealed).
- Strings in C# are immutable. This means that once you have created a string object, and given it a value, this value cannot be changed.

C# - Sealed Class

- At the opposite end of the spectrum from abstract classes are sealed classes.
- While you must derive from an abstract class, you cannot derive from a sealed class.
- It therefore follows that you cannot use both of these together.
- An example in .NET framework class library of a sealed class is `System.String`
- *Why would you make a class sealed?*

C# - String Operations

- During lecture we will cover some examples of string operations
- If you want a string object that can be changed, use the `StringBuilder` class instead.

C# - Error Handling

- Similar to Java, C# uses exceptions to handle problems that occur during the execution of a program. These are not errors, but situations that arise that cannot be controlled, such as trying to read an non-existing file.
- An exception is itself an object of type `System.Exception`.

C# - Error Handling

- Exceptions are thrown by the runtime when there is a problem, but you can also throw your own exceptions with the keyword “throw”.
- When an exception is thrown the runtime looks for a handler in the current method. If there is no immediate handler it goes up the stack of calling methods until a handler is found. If no exception handler is found, the program terminates.

C# - Error Handling

- The try, catch (and finally) syntax is used to define an exception handler.
- You can have multiple exception handlers but they must be placed in order of the most specific to least specific exception type (to be handled).

C# - Error Handling

- C# allows you to create your own exceptions
- What is the need for this?
- Is it good/bad?

SEGMENT 3

GETTING DEEPER INTO OO: C#

C# – Inheritance

- An important OO feature
- What does the child class inherit from parent class?
- Although the methods and variables are inherited from the base class, the constructors are not. Constructors must be defined for each class.
- However you can use `base()` and `this()` with constructors to call the constructor of the base class, or another constructor of the current class respectively.

C# – Inheritance

- The derived class can redefine the methods of the base class.
- In effect, the derived class hides the base class method. This is done by using the keyword new. Of course the derived class can also define its own methods.

C# – Inheritance

- The value of inheritance is that a method that takes a reference to the base class as a parameter, could be passed a reference to the derived class.
- However in this situation, when a method was called using that reference, the method of base class would be invoked, even if the derived class had redefined it.
- This is called *early binding* and we shall examine this problem, and its solution late binding in Polymorphism

C# – Abstract Class

- Abstract classes extend the idea of inheritance and polymorphism. An abstract class can not be instantiated, instead it acts as a “place holder” for common functionality for the derived classes a concept that must be made concrete.
- To define an abstract class at least one method in the class must be made abstract with the keyword `abstract`. The class itself must also use the keyword “`abstract`”.
- The abstract method provides no implementation. Instead, any derived classes must provide the functionality

C# - Interfaces

- Interfaces
 - No instance variables
 - No method definitions
 - Declared with the interface keyword
 - Colon is used on the class to indicate that it inherits the interface
 - Class must then implement the method(s)
- Provides for multiple inheritance
- Provides for inheritance across hierarchies

C# – JSON Introduction

- JSON stands for JavaScript Object Notation.
- It is used for
 - storing and/or
 - exchanging of data.
- The light-weight nature of JSON makes it a very popular choice among the developers.
- It is "language-independent" and it is very easy to understand.
- The friendly semantical-nature of JSON makes it easy to use and understand.
- This format was originally created from JavaScript language by Douglas Crockford.
- The format is actually textual and as such anyone can use it freely and easily.

C# – JSON Introduction

- Here is a simple JSON document-

```
{  
  "burn-mansion":  
  {  
    "burns": "Smithers, release the  
             hounds!",  
    "smithers": "Yes Sir!"  
  }  
}
```

C# & JSON (.NET Core native lib)

- Brand new library: System.Text.Json
- Comes in built as of .NET Core 3.0
- Provides reading / writing among other useful operations
- Official page:
 - <https://learn.microsoft.com/en-au/dotnet/api/system.text.json?view=net-10.0>

C# - LINQ & JSON (3rd party)

- Newtonsoft.Json
- Download via NuGet
- Add using statement Newtonsoft.Json
- Easier to use
- Example

